

UNIT-II

Regression and Classification Models: Linear Regression and Multivariate Regression, Linear Classification, Logistic Regression, Principal Component Analysis and Linear Discriminant Analysis, Multiclass Classification, Naïve-Bayes, Bayesian Networks, Support Vector Machines.

Course Objectives: The students will learn the following

1. Understand the fundamentals of Machine Learning and its various learning paradigms.
2. Comprehend the Statistical Decision Theory and its application in Regression and Classification models.
3. Analyze the Bias-Variance trade-off and its significance in Machine Learning models.
4. Build and interpret Regression and Classification models such as Linear Regression, Logistic Regression, and Naïve-Bayes.
5. Understand and implement different clustering algorithms such as Partitional Clustering, Hierarchical Clustering, and Density-based Clustering.

Course Outcomes: Upon the successful completion of the course, the students will able to:

- CO1. Apply the basic principles and concepts of Machine Learning and its various learning paradigms.
- CO2. Apply Statistical Decision Theory in Regression and Classification models to make informed decisions.
- CO3. Implement Bias-Variance trade-off and its importance in Machine Learning models.
- CO4. Interpret Regression and Classification models such as Linear Regression, Logistic Regression, and Naïve-Bayes.
- CO5. Implement clustering algorithms such as Partitional Clustering, Hierarchical Clustering, and Density-based Clustering, and develop an understanding of their applicability in real-world scenarios.

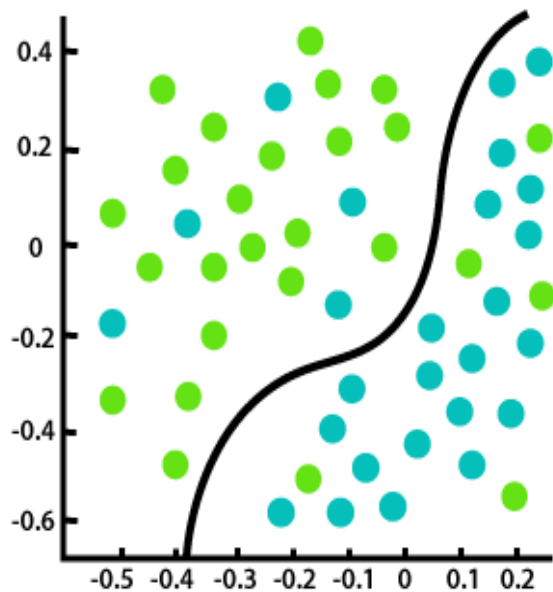
Learning Resources

Textbooks	Unit I	Unit II	Unit III	Unit IV	Unit V
1. Introduction to Machine Learning, Ethem Alpaydin 2nd Edition, MIT Press 2000.	C	C	C	C	C
2. Machine Learning, Tom M. Mitchell, McGraw hill, 1997, ISBN: 0-07-042807-7.			C	C	C

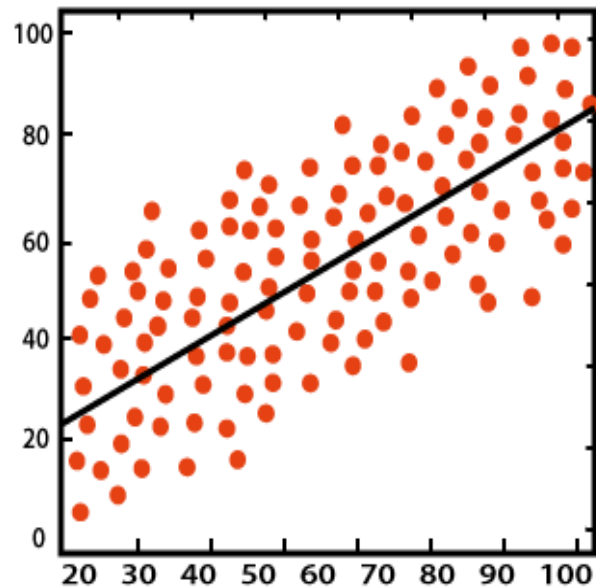
Unit II

Regression and Classification Models

Regression and classification are two fundamental types of supervised machine learning models used in predictive analytics. They serve different purposes and are applied in various domains depending on the nature of the data and the problem at hand.



Classification



Regression

1. Regression Models:

Regression models are used when the target variable (the variable you're trying to predict) is continuous. These models predict a continuous value, such as a price, temperature, or sales volume. Common types of regression models include:

Linear Regression: This is one of the simplest regression algorithms where the relationship between the input variables and the target variable is assumed to be linear. It tries to fit a straight line to the data.

Polynomial Regression: In cases where the relationship between the variables isn't linear, polynomial regression fits a polynomial function to the data.

Ridge Regression, Lasso Regression, Elastic Net: These are regularization techniques used to prevent overfitting in regression models.

Support Vector Regression (SVR): It's an extension of support vector machines for regression tasks.

Decision Tree Regression: Decision trees can also be used for regression tasks, where the target variable is predicted by learning simple decision rules inferred from the data features.

Example: Suppose we want to do weather forecasting, so for this, we will use the Regression algorithm. In weather prediction, the model is trained on the past data, and once the training is completed, it can easily predict the weather for future days.

2. Classification Models:

Classification models are used when the target variable is categorical, i.e., it belongs to a finite set of classes. These models predict the class labels for new instances. Common types of classification models include:

Logistic Regression: Despite its name, logistic regression is a classification algorithm used to model the probability of a binary outcome. It's commonly used for binary classification problems.

Decision Trees: Decision trees can be used for both regression and classification tasks. In classification, they split the data based on features to create distinct classes.

Random Forest: Random forest is an ensemble learning method that builds multiple decision trees and merges their predictions to improve accuracy and prevent overfitting.

Support Vector Machines (SVM): SVMs can be used for both regression and classification tasks. In classification, they try to find the hyperplane that best separates the classes.

K-Nearest Neighbors (KNN): KNN is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (e.g., distance functions).

Example: The best example to understand the Classification problem is Email Spam Detection. The model is trained on the basis of millions of emails on different parameters, and whenever it receives a new email, it identifies whether the email is spam or not. If the email is spam, then it is moved to the Spam folder.

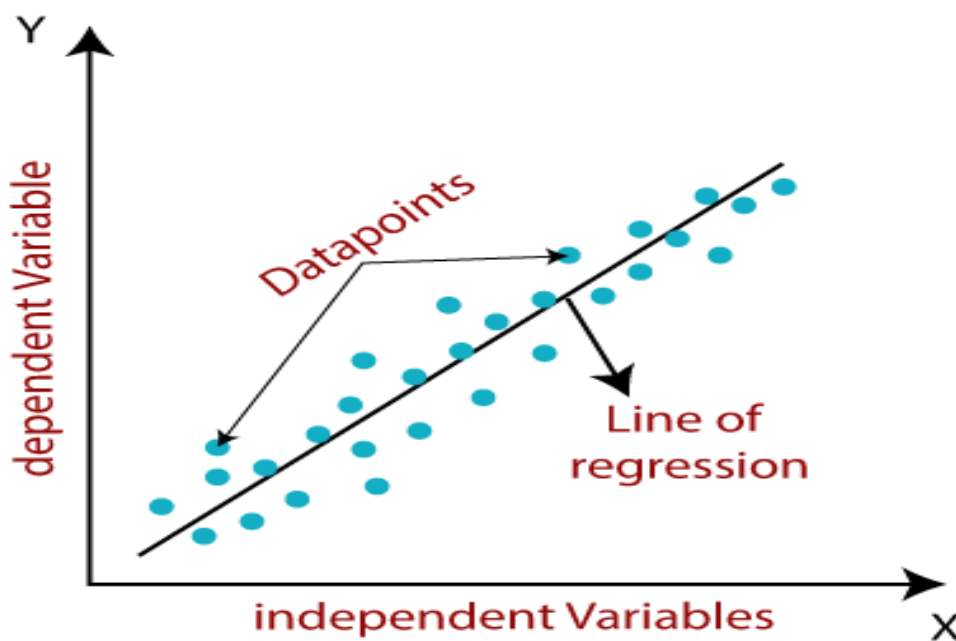
Difference between Regression and Classification

Regression Algorithm	Classification Algorithm
In Regression, the output variable must be of continuous nature or real value.	In Classification, the output variable must be a discrete value.
The task of the regression algorithm is to map the input value (x) with the continuous output variable(y).	The task of the classification algorithm is to map the input value(x) with the discrete output variable(y).
Regression Algorithms are used with continuous data.	Classification Algorithms are used with discrete data.
In Regression, we try to find the best fit line, which can predict the output more accurately.	In Classification, we try to find the decision boundary, which can divide the dataset into different classes.
Regression algorithms can be used to solve the regression problems such as Weather Prediction, House price prediction, etc.	Classification Algorithms can be used to solve classification problems such as Identification of spam emails, Speech Recognition, Identification of cancer cells, etc.
The regression Algorithm can be further divided into Linear and Non-linear Regression.	The Classification algorithms can be divided into Binary Classifier and Multi-class Classifier.

1.Linear Regression

Linear regression is a statistical method used to model the relationship between variables. It assumes a linear relationship between the dependent variable and one or more independent variables. The goal is to find the best-fitting line that minimizes the difference between observed and predicted values. It's widely used for prediction, forecasting, and understanding relationships between variables.

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Types of Linear Regression

Linear regression can be further divided into two types of the algorithm:

a) Simple Linear Regression:

If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

b) Multiple Linear regression:

If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

The basic form of linear regression equation for a single independent variable is:

$$y = a_0 + a_1x + \varepsilon$$

Y= Dependent Variable (Target Variable)

X= Independent Variable (predictor Variable)

a_0 = intercept of the line (Gives an additional degree of freedom)

a_1 = Linear regression coefficient (scale factor to each input value).

ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.

Linear regression can be extended to multiple independent variables, resulting in the multiple linear regression equation:

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \dots + \beta_nX_n + \varepsilon$$

Where $X_1, X_2, \dots, X_n$ are the independent variables, and $\beta_1, \beta_2, \dots, \beta_n$ are their respective coefficients.

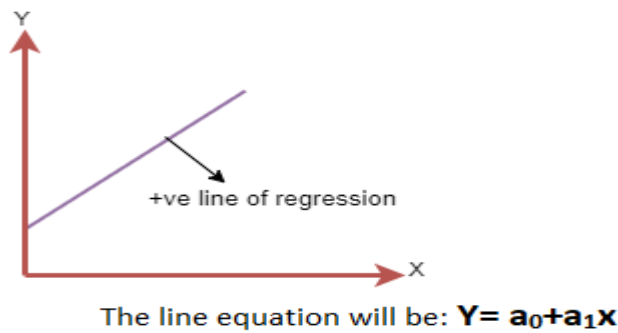
Linear regression is widely used for various purposes such as prediction, forecasting, and understanding the relationship between variables. It is a fundamental technique in statistics and machine learning. However, it's important to note that linear regression assumes a linear relationship between variables, which might not always hold true in real-world scenarios.

Linear Regression Line

A linear line showing the relationship between the dependent and independent variables is called a regression line. A regression line can show two types of relationship:

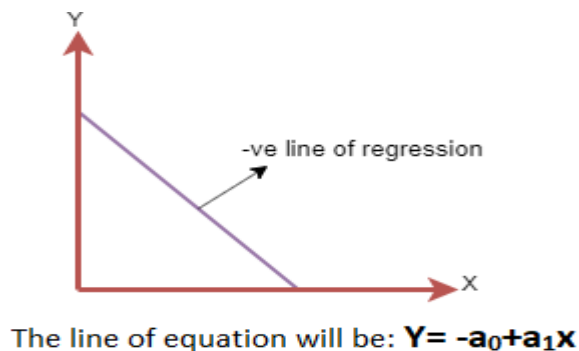
❖ Positive Linear Relationship:

If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.



❖ Negative Linear Relationship:

If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.



Finding the best fit line:

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0 , a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0 , a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the Mean Squared Error (MSE) cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

For the above linear equation, MSE can be calculated as:

$$MSE = \frac{1}{N} \sum_{i=1}^n (y_i - (a_1 x_i + a_0))^2$$

Where,

N=Total number of observation

Y_i = Actual value

$(a_1 x_i + a_0)$ = Predicted value.

Residuals:

The distance between the actual value and predicted values is called residual. If the observed points are far from the regression line, then the residual will be high, and so cost function will high. If the scatter points are close to the regression line, then the residual will be small and hence the cost function.

Gradient Descent:

- Gradient descent is used to minimize the MSE by calculating the gradient of the cost function.
- A regression model uses gradient descent to update the coefficients of the line by reducing the cost function.
- It is done by a random selection of values of coefficient and then iteratively update the values to reach the minimum cost function.

Model Performance:

The Goodness of fit determines how the line of regression fits the set of observations. The process of finding the best model out of various models is called optimization. It can be achieved by below method:

1. R-squared method:

- ❖ R-squared is a statistical method that determines the goodness of fit.
- ❖ It measures the strength of the relationship between the dependent and independent variables on a scale of 0-100%.
- ❖ The high value of R-square determines the less difference between the predicted values and actual values and hence represents a good model.

- ❖ It is also called a coefficient of determination, or coefficient of multiple determination for multiple regression.

It can be calculated from the below formula:

$$R\text{-squared} = \frac{\text{Explained variation}}{\text{Total Variation}}$$

Assumptions of Linear Regression

These are some formal checks while building a Linear Regression model, which ensures to get the best possible result from the given dataset.

Linear relationship between the features and target:

Linear regression assumes the linear relationship between the dependent and independent variables.

Small or no multicollinearity between the features:

Multicollinearity means high-correlation between the independent variables. Due to multicollinearity, it may difficult to find the true relationship between the predictors and target variables. Or we can say, it is difficult to determine which predictor variable is affecting the target variable and which is not. So, the model assumes either little or no multicollinearity between the features or independent variables.

Homoscedasticity Assumption:

Homoscedasticity is a situation when the error term is the same for all the values of independent variables. With homoscedasticity, there should be no clear pattern distribution of data in the scatter plot.

Normal distribution of error terms:

Linear regression assumes that the error term should follow the normal distribution pattern. If error terms are not normally distributed, then confidence intervals will become either too wide or too narrow, which may cause difficulties in finding coefficients. It can be checked using the q-q plot. If the plot shows a straight line without any deviation, which means the error is normally distributed.

No autocorrelations:

The linear regression model assumes no autocorrelation in error terms. If there will be any correlation in the error term, then it will drastically reduce the accuracy of the model. Autocorrelation usually occurs if there is a dependency between residual errors.

2.Multivariate Regression

Multivariate regression refers to the statistical technique that establishes a relationship between multiple data variables. It estimates a linear equation that facilitates the analysis of multiple dependent or outcome variables depending on one or more predictor variables at different points in time.

In multivariate regression, the goal is to find the relationship between the dependent variable Y and multiple independent variables $X_1, X_2, \dots, X_n$. The relationship can be represented by the following equation:

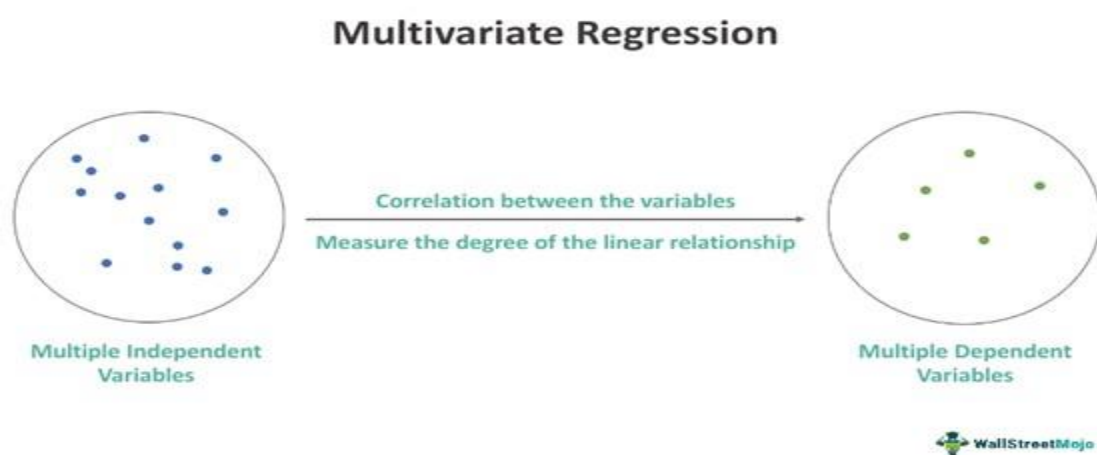
$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

where:

- Y is the dependent variable (the variable we want to predict).
- $X_1, X_2, \dots, X_n$ are the independent variables.
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are the coefficients that represent the strength and direction of the relationship between the independent variables and the dependent variable.
- ϵ is the error term, representing the difference between the observed value of Y and the value predicted by the model.

The coefficients

$\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are estimated from the data using techniques such as ordinary least squares (OLS) regression. The goal is to find the values of the coefficients that minimize the sum of squared differences between the observed values of Y and the values predicted by the model.



- Multivariate regression is a statistical model that predicts multiple dependent variables using two or more independent variables, allowing for a better analysis of interrelated variables through a linear equation.
- The validity and reliability of such a model rely upon the assumptions of independence, linearity, normality, and homoscedasticity.
- It differs from a multiple regression that deals with a single dependent variable and numerous independent variables.
- It can be distinguished from univariate and linear regression since the former deals with a single predictor variable and one response variable, and the latter is a comprehensive approach that includes both univariate and multivariate regression.

Assumptions

The validity and reliability of the multivariate regression findings depend upon the following four assumptions:

1. **Linearity:** The correlation between the predictor and outcome variables is linear.
2. **Independence:** The observations are autonomous of each other, i.e., the value of the other independent variable should not influence the value of the independent variables.
3. **Homoscedasticity:** The variance of the errors (residuals) is even across all levels of the explanatory variables. This ensures that the spread of residuals is the same for all predicted values.
4. **Normality:** The residuals (differences between observed and predicted values) should be normally distributed, ensuring that statistical inferences about regression coefficients are valid.

Formula

The multivariate regression equation is represented as follows:

$$Y = \beta_0 + \beta_1 X_1 + \beta_k X_k + \text{residual}$$

- Y represents the dependent variable.
- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_k$ are the coefficients for the respective independent variables $X_1, X_2, \dots, X_k$.
- Residual represents the error term

Advantages And Disadvantages

The researchers need to consider the following pros and cons before conducting the multivariate regression analysis:

Advantages

Some of the benefits of this model are discussed below:

- **Better Comprehends Relationships:** Unlike simple linear regression, which considers only one predictor, multivariate regression can account for interactions and interdependencies among various predictors, capturing complex relationships between these variables.
- **Reliable Predictions:** By including multiple predictors, the model might provide more accurate estimations than simple regression models, leading to a better fit for the data.
- **Correlation, Strength, and Direction:** Multivariate regression can help identify which explanatory variables significantly influence the dependent variable, establishing a correlation and quantifying the direction and strength of these correlations.

Disadvantages

The various limitations of this regression technique are as follows:

- **Difficult to Interpret:** Multivariate regression can be challenging to interpret, especially for individuals unfamiliar with [statistical analyses](#), due to multiple predictors.
- **Complex Calculations:** Since this model incorporates multiple variables, its computation involves complex mathematical calculations.
- **Extensive Data Requirement:** Multivariate regression requires a larger [sample size](#) than simple regression. Small sample sizes can result in unreliable parameter estimates and low statistical power.
- **Overfitting:** It occurs when the model fits the training data too closely, capturing noise rather than the underlying pattern.

How to use Multivariate Regression Analysis?

The processes involved in multivariate regression analysis include the selection of features, engineering the features, feature normalization, selection loss functions, hypothesis analysis, and creating a regression model.

- **Selection of features:** It is the most important step in multivariate regression. Also known as variable selection, this process involves selecting viable variables to build efficient models.
- **Feature Normalizing:** This involves feature scaling to maintain streamlined distribution and data ratios. This helps in better data analysis. The value of all the features can be changed according to the requirement.
- **Selecting Loss function and hypothesis:** The loss function is used for predicting errors. The loss function comes into play when the hypothesis prediction changes from the actual figures. Here, the hypothesis represents the value predicted from the feature or variable.
- **Fixing hypothesis parameter:** The parameter of the hypothesis is fixed or set in such a way that it minimizes the loss function and enhances better prediction.
- **Reducing the loss function:** The loss function is minimized by generating an algorithm specifically for loss minimization on the dataset which in turn facilitates the alteration of hypothesis parameters. Gradient descent is the most commonly used algorithm for loss minimization. The algorithm can also be used for other actions once the loss minimization is complete.
- **Analyzing the hypothesis function:** The function of the hypothesis needs to be analyzed as it is crucial for predicting the values. After the function is analyzed, it is then tested on test data.

3.Linear Classification

Classification Algorithm in Machine Learning

Classification algorithms are a subset of supervised learning techniques. Their primary purpose is to predict the category or class of new observations based on training data.

Unlike regression algorithms that predict continuous values, classification algorithms deal with categorical outcomes.

Examples of classification problems include:

Email Spam Detection: Classifying emails as spam or not spam.

Gender Prediction: Determining whether a given name corresponds to a male or female.

Image Recognition: Identifying objects in images (e.g., cat, dog, car).

The output of a classification algorithm is a category, not a numerical value.

There are two main types of classification:

Binary Classifier: When there are only two possible outcomes (e.g., yes/no, spam/not spam).

Multi-class Classifier: When there are more than two possible outcomes (e.g., types of crops, music genres).

Learners in Classification Problems:

Lazy Learners: These algorithms store the training dataset and classify test data based on related stored data. Examples include k-nearest neighbors (K-NN) and case-based reasoning.

Eager Learners: These algorithms build a classification model before receiving test data. Examples include decision trees, Naïve Bayes, and artificial neural networks (ANN).

Linear Classification Boundaries:

- Linear classifiers use a line, plane, or hyperplane to separate data into distinct classes.
- The dividing boundary is linear with respect to the input space.
- Binary linear classifiers create a straight line (or hyperplane) to separate two classes.

Example: Linear Discriminant Analysis (LDA):

- LDA is a linear classification technique that finds the best linear combination of features to separate classes.
- It's commonly used for dimensionality reduction and feature extraction.
- LDA aims to maximize the distance between class means while minimizing the variance within each class.

Types of ML Classification Algorithms:

Classification Algorithms can be further divided into the Mainly two category:

Linear Models

- Logistic Regression
- Support Vector Machines

Non-linear Models

- K-Nearest Neighbours
- Kernel SVM
- Naïve Bayes
- Decision Tree Classification
- Random Forest Classification

Evaluating a Classification model:

Once our model is completed, it is necessary to evaluate its performance; either it is a Classification or Regression model. So for evaluating a Classification model, we have the following ways:

1. Log Loss or Cross-Entropy Loss:

- It is used for evaluating the performance of a classifier, whose output is a probability value between the 0 and 1.
- For a good binary Classification model, the value of log loss should be near to 0.
- The value of log loss increases if the predicted value deviates from the actual value.
- The lower log loss represents the higher accuracy of the model.

2. Confusion Matrix:

- The confusion matrix provides us a matrix/table as output and describes the performance of the model.
- It is also known as the error matrix.
- The matrix consists of predictions result in a summarized form, which has a total number of correct predictions and incorrect predictions. The matrix looks like as below table:

	Actual Positive	Actual Negative
Predicted Positive	True Positive	False Positive
Predicted Negative	False Negative	True Negative

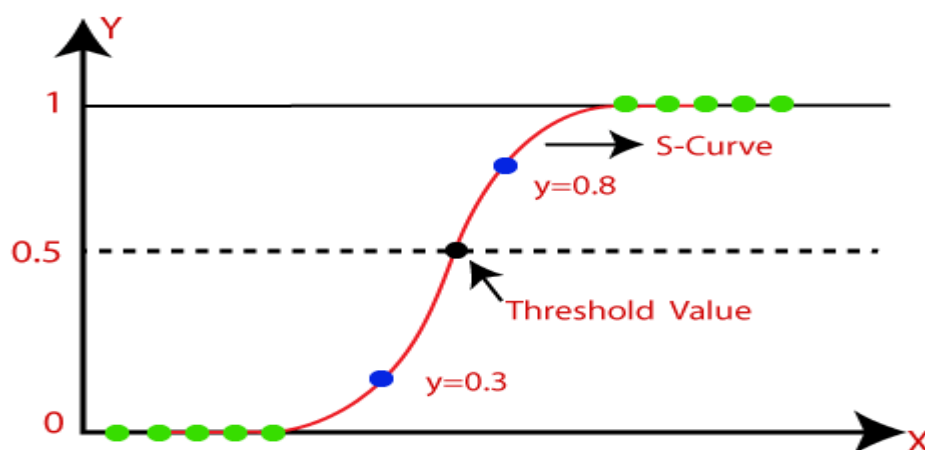
$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Population}}$$

3. AUC-ROC curve:

- ROC curve stands for **Receiver Operating Characteristics Curve** and AUC stands for **Area Under the Curve**.
- It is a graph that shows the performance of the classification model at different thresholds.
- To visualize the performance of the multi-class classification model, we use the AUC-ROC Curve.
- The ROC curve is plotted with TPR and FPR, where TPR (True Positive Rate) on Y-axis and FPR (False Positive Rate) on X-axis.

4.Logistic Regression

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function:



Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.

Logistic Regression Equation:

The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}; 0 \text{ for } y=0, \text{ and infinity for } y=1$$

- But we need range between -[infinity] to +[infinity], then take logarithm of the equation it will become:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

The above equation is the final equation for Logistic Regression.

Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

Steps in Logistic Regression:

- Data Pre-processing step
- Fitting Logistic Regression to the Training set
- Predicting the test result
- Test accuracy of the result (Creation of Confusion matrix)
- Visualizing the test set result.

5. Principal Component Analysis

Principal Component Analysis (PCA) is a dimensionality reduction technique commonly used in machine learning and data analysis. Its primary goal is to reduce the dimensionality of a dataset while preserving as much of its variance as possible.

How PCA works using an example:

Suppose you have a dataset with several features or dimensions. For instance, let's say you have a dataset that contains information about houses, such as size (in square feet), number of bedrooms, number of bathrooms, and price. Each house in the dataset is represented as a point in a high-dimensional space, with each feature being one dimension.

PCA aims to find a new set of dimensions, called principal components, that capture the most significant variability in the data. These principal components are linear combinations of the original features.

PCA Steps:

1. **Standardize the Data:** It's essential to standardize the features by subtracting the mean and scaling to unit variance. This step ensures that all features contribute equally to the analysis.

First, we need to standardize our dataset to ensure that each variable has a mean of 0 and a standard deviation of 1.

$$Z = \frac{X - \mu}{\sigma}$$

- μ is the mean of independent features

$$\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$$

- is the standard deviation of independent features

$$\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$$

2. **Compute the Covariance Matrix:** Next, compute the covariance matrix of the standardized data. The covariance matrix indicates the relationships between the different features in the dataset.

Covariance measures the strength of joint variability between two or more variables, indicating how much they change in relation to each other. To find the covariance we can use the formula:

$$\text{cov}(x1, x2) = \frac{\sum_{i=1}^n (x1_i - \bar{x1})(x2_i - \bar{x2})}{n-1}$$

The value of covariance can be positive, negative, or zeros.

- Positive: As the x1 increases x2 also increases.
- Negative: As the x1 increases x2 also decreases.
- Zeros: No direct relation

3. **Compute Eigenvectors and Eigenvalues:** Calculate the eigenvectors and eigenvalues of the covariance matrix. Eigenvectors represent the directions of maximum variance in the data, while eigenvalues indicate the magnitude of variance along those directions.

Let A be a square nXn matrix and X be a non-zero vector for which

$$AX = \lambda X$$

for some scalar values λ . then λ is known as the eigenvalue of matrix A and X is known as the eigenvector of matrix A for the corresponding eigenvalue.

It can also be written as :

$$AX - \lambda X = 0$$

$$(A - \lambda I)X = 0$$

where I am the identity matrix of the same shape as matrix A. And the above conditions will be true only if $(A - \lambda I)$ will be non-invertible (i.e. singular matrix). That means,

$$|A - \lambda I| = 0$$

From the above equation, we can find the eigenvalues λ , and therefore corresponding eigenvector can be found using the equation $|A - \lambda I| = 0$

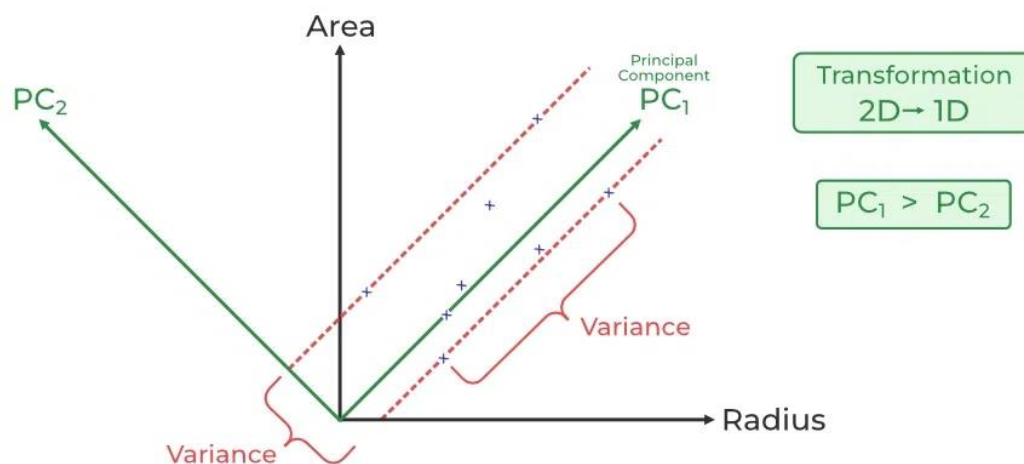
4. **Select Principal Components:** Sort the eigenvectors based on their corresponding eigenvalues in descending order. The eigenvectors with the highest eigenvalues (explained variance) are the principal components.
5. **Project Data onto Principal Components:** Finally, project the original data onto the new feature space formed by the selected principal

components. This results in a lower-dimensional representation of the data while retaining most of its variance.

In That example, after performing PCA, you might find that the first principal component captures most of the variability in the dataset. This component could represent a combination of features that correlate strongly with the overall size or value of the houses. Subsequent principal components might capture additional, smaller sources of variance, such as specific characteristics like the number of bedrooms versus the number of bathrooms.

PCA can be useful for various tasks, such as visualization, noise reduction, feature extraction, and speeding up machine learning algorithms by reducing the dimensionality of the data.

Principal Component Analysis (PCA) is used to reduce the dimensionality of a data set by finding a new set of variables, smaller than the original set of variables, retaining most of the sample's information, and useful for the [regression and classification](#) of data.



1. Principal Component Analysis (PCA) is a technique for dimensionality reduction that identifies a set of orthogonal axes, called principal components, that capture the maximum variance in the data. The principal components are linear combinations of the original variables in the dataset and are ordered in decreasing order of importance. The total variance captured by all the principal components is equal to the total variance in the original dataset.

2. The first principal component captures the most variation in the data, but the second principal component captures the maximum [variance](#) that is [orthogonal](#) to the first principal component, and so on.
3. Principal Component Analysis can be used for a variety of purposes, including data visualization, feature selection, and data compression. In data visualization, PCA can be used to plot high-dimensional data in two or three dimensions, making it easier to interpret. In feature selection, PCA can be used to identify the most important variables in a dataset. In data compression, PCA can be used to reduce the size of a dataset without losing important information.
4. In Principal Component Analysis, it is assumed that the information is carried in the variance of the features, that is, the higher the variation in a feature, the more information that features carries.

Applications of PCA:

- **Data visualization:** Plotting data in 2D or 3D for better understanding.
- **Feature selection:** Identifying the most important features.
- **Noise reduction:** Removing noisy features.
- **Compression:** Reducing storage requirements.

6.Linear Discriminant Analysis

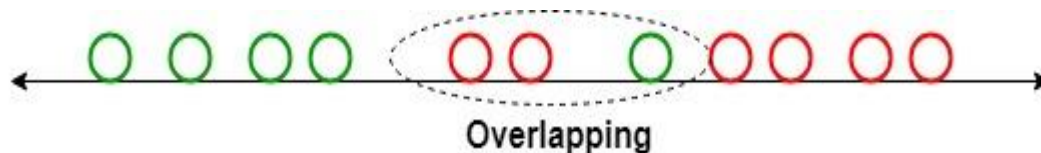
Linear Discriminant Analysis (LDA) is a classification technique used in machine learning and statistics. It's a supervised learning algorithm that's commonly employed for dimensionality reduction and classification tasks.

Linear Discriminant Analysis (LDA), also known as Normal Discriminant Analysis or Discriminant Function Analysis, is a dimensionality reduction technique primarily utilized in supervised classification problems.

It facilitates the modelling of distinctions between groups, effectively separating two or more classes. LDA operates by projecting features from a higher-dimensional space into a lower-dimensional one.

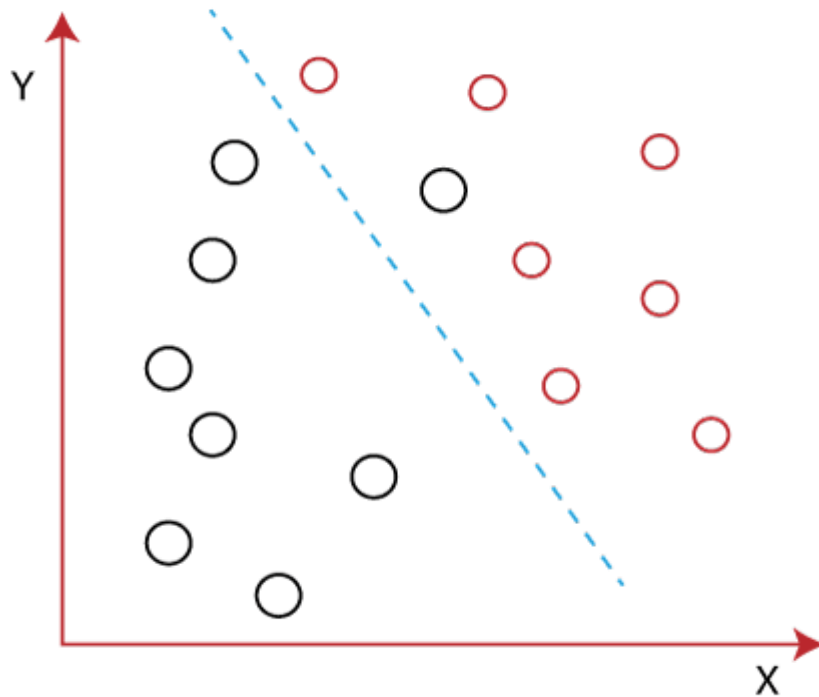
In machine learning, LDA serves as a supervised learning algorithm specifically designed for classification tasks, aiming to identify a linear combination of features that optimally segregates classes within a dataset.

For example, we have two classes and we need to separate them efficiently. Classes can have multiple features. Using only a single feature to classify them may result in some overlapping as shown in the below figure. So, we will keep on increasing the number of features for proper classification.



Example:

Let's assume we have to classify two different classes having two sets of data points in a 2-dimensional plane as shown below image:



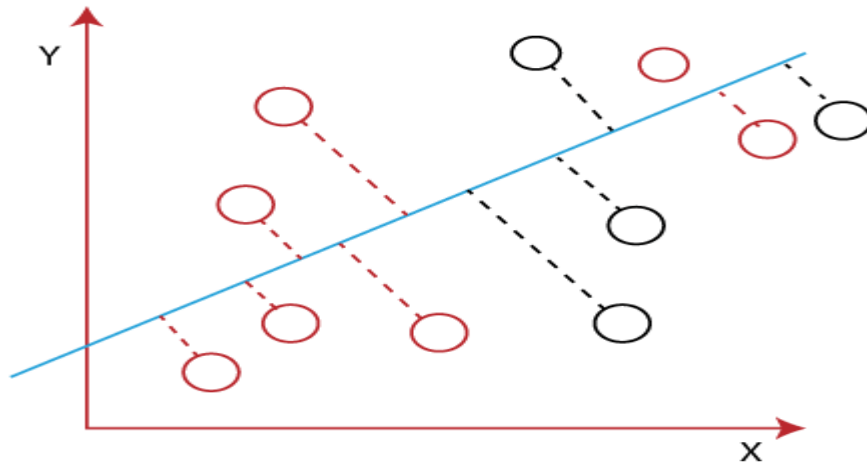
it is impossible to draw a straight line in a 2-d plane that can separate these data points efficiently but using linear Discriminant analysis; we can dimensionally reduce the 2-D plane into the 1-D plane. Using this technique, we can also maximize the separability between multiple classes.

How Linear Discriminant Analysis (LDA) works?

Linear Discriminant analysis is used as a dimensionality reduction technique in machine learning, using which we can easily transform a 2-D and 3-D graph into a 1-dimensional plane.

Let's consider an example where we have two classes in a 2-D plane having an X-Y axis, and we need to classify them efficiently. As we have already seen in the above example that LDA enables us to draw a straight line that can completely separate the two classes of the data points. Here, LDA uses an X-Y axis to create a new axis by separating them using a straight line and projecting data onto a new axis.

Hence, we can maximize the separation between these classes and reduce the 2-D plane into 1-D.



To create a new axis, Linear Discriminant Analysis uses the following criteria:

- It maximizes the distance between means of two classes.
- It minimizes the variance within the individual class.

Using the above two conditions, LDA generates a new axis in such a way that it can maximize the distance between the means of the two classes and minimizes the variation within each class.

In other words, we can say that the new axis will increase the separation between the data points of the two classes and plot them onto the new axis.

Linear Discriminant Analysis (LDA) is a powerful statistical technique used for classification and dimensionality reduction. Here's why it's widely used:

1. **Dimensionality Reduction:** LDA helps in reducing the dimensionality of the feature space while preserving the discriminatory information between classes. It projects the data onto a lower-dimensional space, making it easier to visualize and analyze while retaining as much discriminatory information as possible.
2. **Classification:** LDA is a supervised learning algorithm, meaning it requires labeled data. It finds the linear combination of features that best separates the classes. This makes it an effective tool for classification tasks where the classes are well-defined and separable.
3. **Assumption of Normality:** LDA assumes that the features within each class are normally distributed. While this assumption may not always hold true, LDA can still perform reasonably well even when the assumption is violated, especially with large sample sizes.

4. **Optimal Separation:** LDA aims to find the projection that maximizes the between-class variance while minimizing the within-class variance. This results in a projection that maximizes the class separability, making it an optimal choice for classification tasks.
5. **Low Computational Cost:** LDA has a relatively low computational cost compared to other classification algorithms, such as Support Vector Machines (SVM) or Neural Networks. This makes it suitable for large datasets and real-time applications.
6. **Robustness to Overfitting:** LDA tends to be less prone to overfitting, especially when the number of samples is small compared to the number of features. It achieves this by regularizing the covariance matrices involved in the calculation.
7. **Interpretability:** The linear nature of LDA makes the results easy to interpret. The coefficients obtained from LDA can provide insights into the importance of different features in discriminating between classes.

Extension to Linear Discriminant Analysis (LDA)

1. **Quadratic Discriminant Analysis (QDA):** For multiple input variables, each class deploys its own estimate of variance.
2. **Flexible Discriminant Analysis (FDA):** it is used when there are non-linear groups of inputs are used, such as splines.
3. **Flexible Discriminant Analysis (FDA):** This uses regularization in the estimate of the variance (actually covariance) and hence moderates the influence of different variables on LDA.

How to Prepare Data for LDA

- **Classification Problems:** LDA is mainly applied for classification problems to classify the categorical output variable. It is suitable for both binary and multi-class classification problems.
- **Gaussian Distribution:** The standard LDA model applies the Gaussian Distribution of the input variables. One should review the univariate distribution of each attribute and transform them into more Gaussian-looking distributions. For e.g., use log and root for exponential distributions and Box-Cox for skewed distributions.
- **Remove Outliers:** It is good to firstly remove the outliers from your data because these outliers can skew the basic statistics used to separate classes in LDA, such as the mean and the standard deviation.
- **Same Variance:** As LDA always assumes that all the input variables have the same variance, hence it is always a better way to firstly standardize the data before implementing an LDA model. By this, the Mean will be 0, and it will have a standard deviation of 1.

7.Multiclass classification

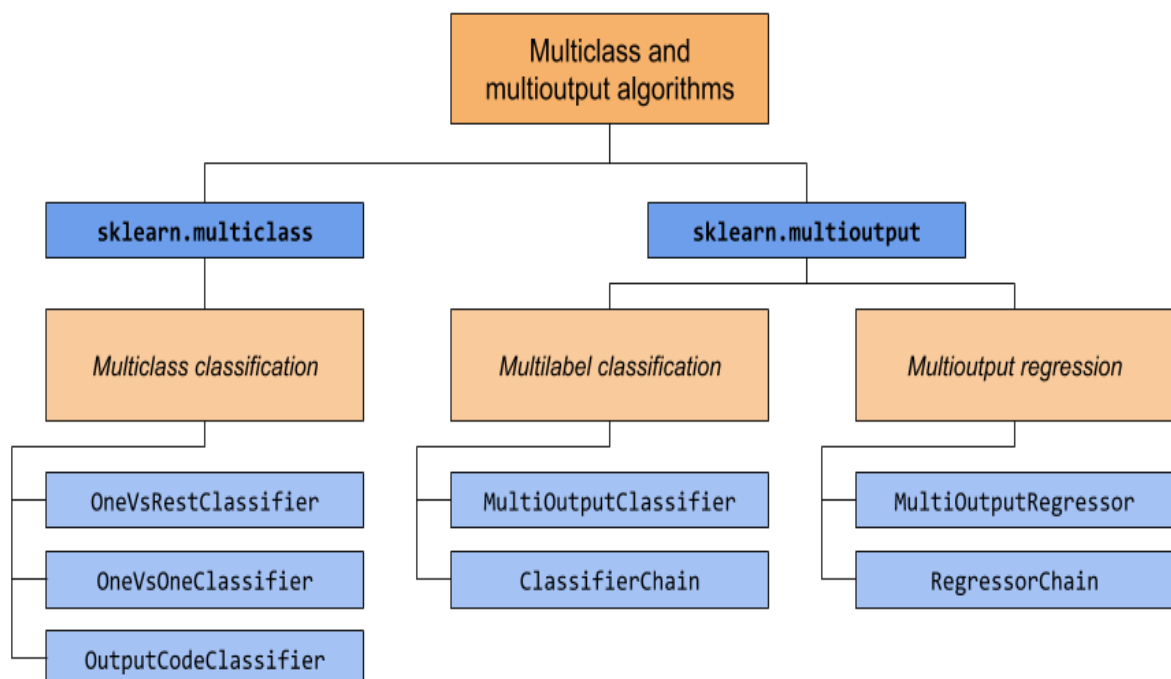
Multiclass classification is a machine learning task where we classify instances into three or more classes. Unlike binary classification, which involves classifying instances into one of two classes, multiclass classification deals with more diverse categories.

What is Multiclass Classification?

In multiclass classification, our goal is to categorize data into multiple classes based on their similarities.

The target variable (dependent variable) has more than two categories.

For example, consider the Iris dataset, which contains three species of Iris plants: Virginica, Setosa, and Versicolor. Each species corresponds to a different class¹.



- Multiclass classification is a classification task with more than two classes. Each sample can only be labeled as one class.
- For example, classification using features extracted from a set of images of fruit, where each image may either be of an orange, an apple, or a pear. Each image is one sample and is labeled as one of the 3 possible classes. Multiclass classification makes the assumption that each

sample is assigned to one and only one label - one sample cannot, for example, be both a pear and an apple.

- While all scikit-learn classifiers are capable of multiclass classification, the meta-estimators offered by `sklearn.multiclass` permit changing the way they handle more than two classes because this may have an effect on classifier performance (either in terms of generalization error or required computational resources)

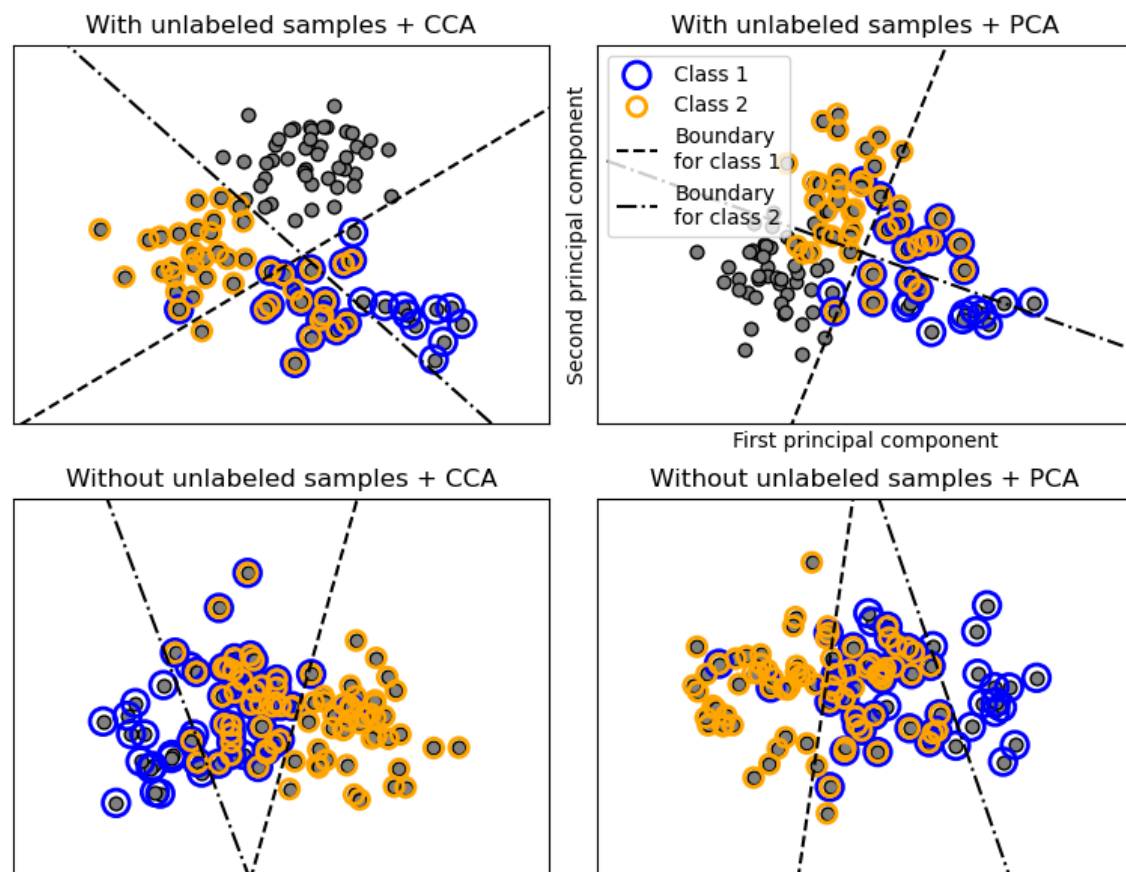
A. One Vs Rest Classifier

The **one-vs-rest** strategy, also known as **one-vs-all**, is implemented in **OneVsRestClassifier**. The strategy consists in fitting one classifier per class. For each classifier, the class is fitted against all the other classes. In addition to its computational efficiency (only `n_classes` classifiers are needed), one advantage of this approach is its interpretability. Since each class is represented by one and only one classifier, it is possible to gain knowledge about the class by inspecting its corresponding classifier. This is the most commonly used strategy and is a fair default choice.

Below is an example of multiclass learning using OvR:

```
>>> from sklearn import datasets
>>> from sklearn.multiclass import OneVsRestClassifier
>>> from sklearn.svm import LinearSVC
>>> X, y = datasets.load_iris(return_X_y=True)
>>> OneVsRestClassifier(LinearSVC(dual="auto", random_state=0)).fit(X,
y).predict(X)
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

OneVsRestClassifier also supports multilabel classification. To use this feature, feed the classifier an indicator matrix, in which cell `[i, j]` indicates the presence of label `j` in sample `i`.



Examples:

- [Multilabel classification](#)

B.One Vs One Classifier

OneVsOneClassifier constructs one classifier per pair of classes. At prediction time, the class which received the most votes is selected. In the event of a tie (among two classes with an equal number of votes), it selects the class with the highest aggregate classification confidence by summing over the pair-wise classification confidence levels computed by the underlying binary classifiers.

Since it requires to fit $n_classes * (n_classes - 1) / 2$ classifiers, this method is usually slower than one-vs-the-rest, due to its $O(n_classes^2)$ complexity. However, this method may be advantageous for algorithms such as kernel algorithms which don't scale well with $n_samples$. This is because each individual learning problem only involves a small subset of the data whereas, with one-vs-the-rest, the complete dataset is used $n_classes$ times. The decision

function is the result of a monotonic transformation of the one-versus-one classification.

Below is an example of multiclass learning using OvO:

```
>>> from sklearn import datasets
>>> from sklearn.multiclass import OneVsOneClassifier
>>> from sklearn.svm import LinearSVC
>>> X, y = datasets.load_iris(return_X_y=True)
>>> OneVsOneClassifier(LinearSVC(dual="auto", random_state=0)).fit(X,
y).predict(X)
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 2, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

C. Output Code Classifier

Error-Correcting Output Code-based strategies are fairly different from one-vs-the-rest and one-vs-one. With these strategies, each class is represented in a Euclidean space, where each dimension can only be 0 or 1. Another way to put it is that each class is represented by a binary code (an array of 0 and 1). The matrix which keeps track of the location/code of each class is called the code book. The code size is the dimensionality of the aforementioned space. Intuitively, each class should be represented by a code as unique as possible and a good code book should be designed to optimize classification accuracy. In this implementation, we simply use a randomly-generated code book as advocated in [3] although more elaborate methods may be added in the future.

At fitting time, one binary classifier per bit in the code book is fitted. At prediction time, the classifiers are used to project new points in the class space and the class closest to the points is chosen.

In OutputCodeClassifier, the code_size attribute allows the user to control the number of classifiers which will be used. It is a percentage of the total number of classes.

A number between 0 and 1 will require fewer classifiers than one-vs-the-rest. In theory, $\log_2(n_classes) / n_classes$ is sufficient to represent each class

unambiguously. However, in practice, it may not lead to good accuracy since $\log_2(n_classes)$ is much smaller than $n_classes$.

A number greater than 1 will require more classifiers than one-vs-the-rest. In this case, some classifiers will in theory correct for the mistakes made by other classifiers, hence the name “error-correcting”. In practice, however, this may not happen as classifier mistakes will typically be correlated. The error-correcting output codes have a similar effect to bagging.

Below is an example of multiclass learning using Output-Codes:

```
>>> from sklearn import datasets
>>> from sklearn.multiclass import OutputCodeClassifier
>>> from sklearn.svm import LinearSVC
>>> X, y = datasets.load_iris(return_X_y=True)
>>> clf = OutputCodeClassifier(LinearSVC(dual="auto", random_state=0),
...                             code_size=2, random_state=0)
>>> clf.fit(X, y).predict(X)
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1,
       1, 2, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 1, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Classifiers for Multiclass Classification:

Several algorithms can be used for multiclass classification:

- **Naïve Bayes:** A parametric algorithm that assumes features are independent (though this assumption is often not true).
- **Decision Trees:** A non-parametric algorithm that splits data based on features.
- **Support Vector Machines (SVM):** A powerful algorithm that finds optimal hyperplanes to separate classes.
- **Random Forest Classifier:** An ensemble method combining multiple decision trees.
- **K-Nearest Neighbors (KNN):** A non-parametric algorithm that classifies based on nearest neighbors.

- **Logistic Regression:** A linear model used for binary and multiclass classification.

Performance Metrics:

To evaluate multiclass classification models, we use metrics like:

- **Entropy:** Measures the impurity of a split in decision trees.
- **Confusion Matrix:** Helps assess model performance by comparing predicted and actual class labels.

Multiclass vs. Multi-label Classification:

- **Multiclass classification:** Each sample belongs to only one class.
- **Multi-label classification:** Each sample can belong to multiple classes simultaneously.

8.Naïve Bayes Algorithm

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.**
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles.**

Why is it called Naïve Bayes?

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of Bayes' Theorem.

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

$P(A|B)$ is Posterior probability: Probability of hypothesis A on the observed event B.

$P(B|A)$ is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

$P(A)$ is Prior Probability: Probability of hypothesis before observing the evidence.

$P(B)$ is Marginal Probability: Probability of Evidence.

Working of Naïve Bayes' Classifier:

Working of Naïve Bayes' Classifier can be understood with the help of the below example:

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions. So to solve this problem, we need to follow the below steps:

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

Problem: If the weather is sunny, then the Player should play or not?

Solution: To solve this, first consider the below dataset:

Outlook		Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes

4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	$5/14 = 0.35$

Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Applying Bayes'theorem:

$$P(\text{Yes}|\text{Sunny})= P(\text{Sunny}|\text{Yes})*P(\text{Yes})/P(\text{Sunny})$$

$$P(\text{Sunny}|\text{Yes})= 3/10= 0.3$$

$$P(\text{Sunny})= 0.35$$

$$P(\text{Yes})=0.71$$

$$\text{So } P(\text{Yes}|\text{Sunny}) = 0.3*0.71/0.35= \mathbf{0.60}$$

$$P(\text{No}|\text{Sunny})= P(\text{Sunny}|\text{No})*P(\text{No})/P(\text{Sunny})$$

$$P(\text{Sunny}|\text{NO})= 2/4=0.5$$

$$P(\text{No})= 0.29$$

$$P(\text{Sunny})= 0.35$$

$$\text{So } P(\text{No}|\text{Sunny})= 0.5*0.29/0.35 = \mathbf{0.41}$$

So as we can see from the above calculation that **$P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$**

Hence on a Sunny day, Player can play the game.

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for **text classification problems**.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Applications of Naïve Bayes Classifier:

- It is used for **Credit Scoring**.
- It is used in **medical data classification**.
- It can be used in **real-time predictions** because Naïve Bayes Classifier is an eager learner.
- It is used in Text classification such as **Spam filtering** and **Sentiment analysis**.

Types of Naïve Bayes Model:

There are three types of Naive Bayes Model, which are given below:

- **Gaussian:** The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial:** The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.
- **Bernoulli:** The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

Python Implementation of the Naïve Bayes algorithm:

Now we will implement a Naive Bayes Algorithm using Python. So for this, we will use the "**user_data**" **dataset**, which we have used in our other classification model. Therefore we can easily compare the Naive Bayes model with the other models.

Steps to implement:

- Data Pre-processing step
- Fitting Naive Bayes to the Training set
- Predicting the test result
- Test accuracy of the result(Creation of Confusion matrix)
- Visualizing the test set result.

9. Bayesian Network

- Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty. We can define a Bayesian network as:
- "A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."
- It is also called a **Bayes network, belief network, decision network, or Bayesian model.**
- Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.

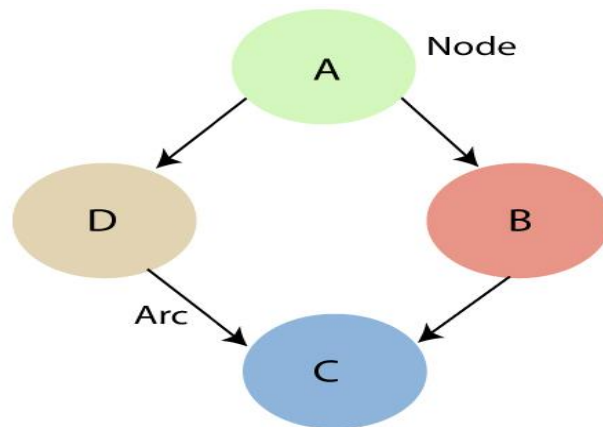
Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network. It can also be used in various tasks including **prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction, and decision making under uncertainty.**

Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

- **Directed Acyclic Graph**
- **Table of conditional probabilities.**

The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram.**

A Bayesian network graph is made up of nodes and Arcs (directed links), where:



- Each **node** corresponds to the random variables, and a variable can be **continuous** or **discrete**.
- **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables. These directed links or arrows connect the pair of nodes in the graph. These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other
 - In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.
 - If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.
 - Node C is independent of node A.

The Bayesian network has mainly two components:

- **Causal Component**
- **Actual numbers**

Each node in the Bayesian network has condition probability distribution $P(X_i | \text{Parent}(X_i))$, which determines the effect of the parent on that node.

Bayesian network is based on Joint probability distribution and conditional probability. So let's first understand the joint probability distribution:

Joint probability distribution:

If we have variables $x_1, x_2, x_3, \dots, x_n$, then the probabilities of a different combination of $x_1, x_2, x_3, \dots, x_n$, are known as Joint probability distribution.

$P[x_1, x_2, x_3, \dots, x_n]$, it can be written as the following way in terms of the joint probability distribution.

$$= P[X_1 | X_2, X_3, \dots, X_n] P[X_2, X_3, \dots, X_n]$$

$$= P[X_1 | X_2, X_3, \dots, X_n] P[X_2 | X_3, \dots, X_n] \dots P[X_{n-1} | X_n] P[X_n].$$

$$\text{Equation : } P(X_i | X_{i-1}, \dots, X_1) = P(X_i | \text{Parents}(X_i))$$

Example:

Solution:

- The Bayesian network for the above problem is given below. The network structure is showing that burglary and earthquake is the parent node of the alarm and directly affecting the probability of alarm's going off, but David and Sophia's calls depend on alarm probability.
- The network is representing that our assumptions do not directly perceive the burglary and also do not notice the minor earthquake, and they also not confer before calling.
- The conditional distributions for each node are given as conditional probabilities table or CPT.
- Each row in the CPT must be sum to 1 because all the entries in the table represent an exhaustive set of cases for the variable.
- In CPT, a boolean variable with k boolean parents contains 2^k probabilities. Hence, if there are two parents, then CPT will contain 4 probability values

List of all events occurring in this network:

- **Burglary (B)**
- **Earthquake(E)**
- **Alarm(A)**
- **David Calls(D)**
- **Sophia calls(S)**

We can write the events of problem statement in the form of probability:

P[D, S, A, B, E], can rewrite the above probability statement using joint probability distribution:

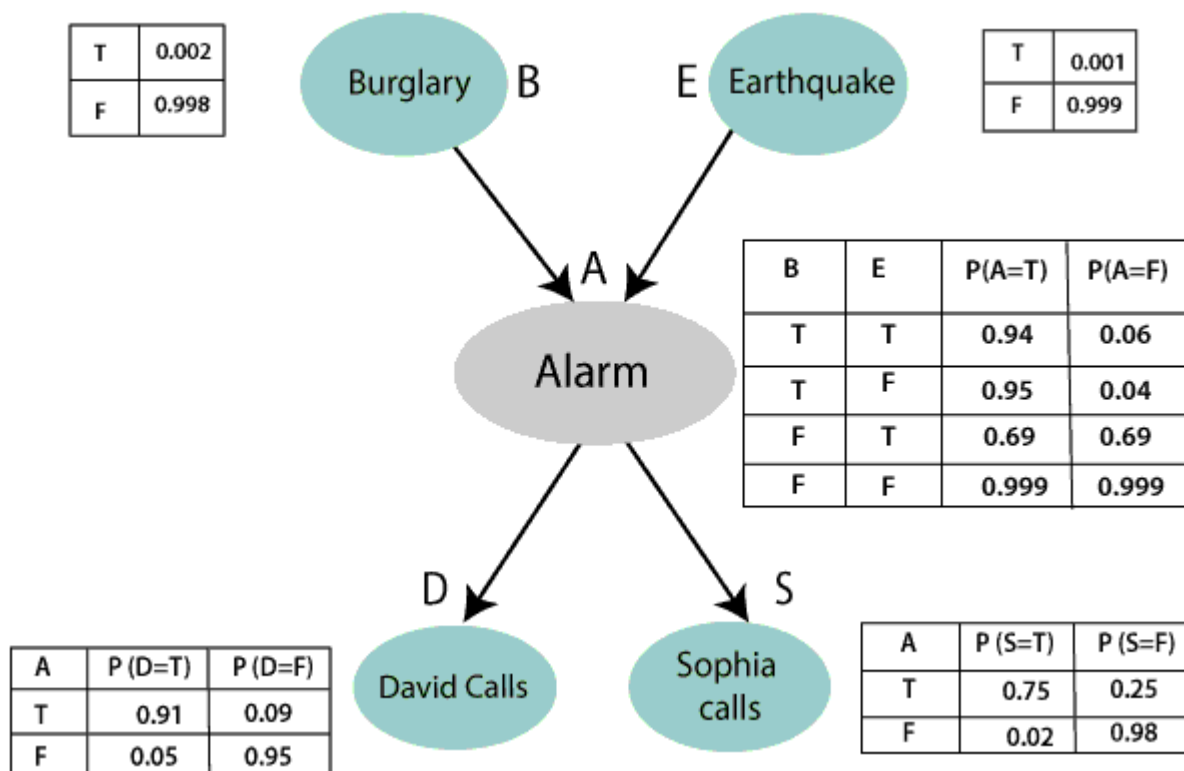
$$P[D, S, A, B, E] = P[D | S, A, B, E] \cdot P[S, A, B, E]$$

$$= P[D | S, A, B, E] \cdot P[S | A, B, E] \cdot P[A, B, E]$$

$$= P[D | A] \cdot P[S | A, B, E] \cdot P[A, B, E]$$

$$= P[D | A] \cdot P[S | A] \cdot P[A | B, E] \cdot P[B, E]$$

$$= P[D | A] \cdot P[S | A] \cdot P[A | B, E] \cdot P[B | E] \cdot P[E]$$



The observed probability for the Burglary and earthquake component:

$P(B = \text{True}) = 0.002$, which is the probability of burglary.

$P(B = \text{False}) = 0.998$, which is the probability of no burglary.

$P(E = \text{True}) = 0.001$, which is the probability of a minor earthquake

$P(E = \text{False}) = 0.999$, Which is the probability that an earthquake not occurred.

Conditional probability table for Alarm A:

The Conditional probability of Alarm A depends on Burglar and earthquake:

B	E	P(A= True)	P(A= False)
True	True	0.94	0.06
True	False	0.95	0.04
False	True	0.31	0.69
False	False	0.001	0.999

Conditional probability table for David Calls:

The Conditional probability of David that he will call depends on the probability of Alarm.

A	P(D= True)	P(D= False)
True	0.91	0.09
False	0.05	0.95

Conditional probability table for Sophia Calls:

The Conditional probability of Sophia that she calls is depending on its Parent Node "Alarm."

A	P(S= True)	P(S= False)
True	0.75	0.25
False	0.02	0.98

The Formula of joint distribution

$$P(S, D, A, \neg B, \neg E) = P(S|A) * P(D|A) * P(A|\neg B \wedge \neg E) * P(\neg B) * P(\neg E).$$

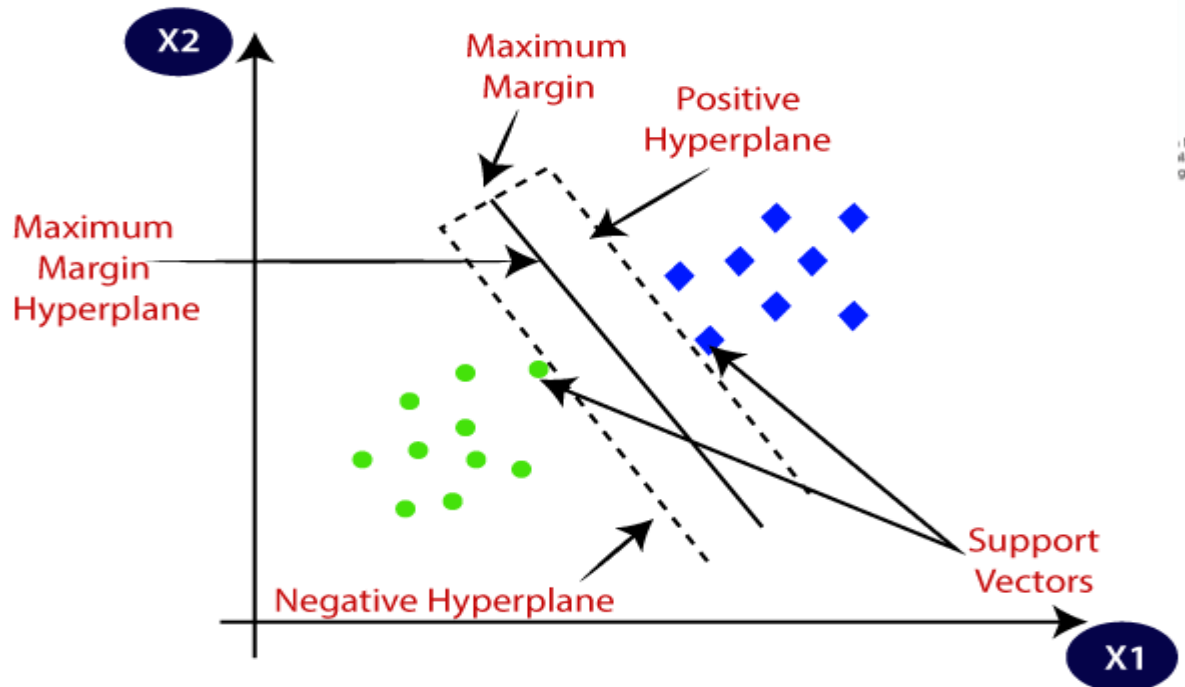
$$= 0.75 * 0.91 * 0.001 * 0.998 * 0.999$$

$$= 0.00068045.$$

10.Support Vector Machines

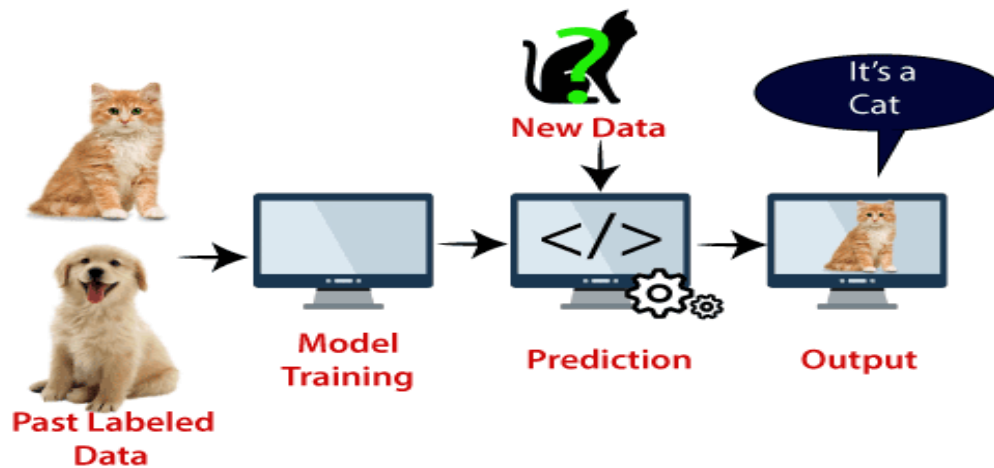
- Support Vector Machine (SVM) is a powerful machine learning algorithm used for linear or nonlinear classification, regression, and even outlier detection tasks.
- SVMs can be used for a variety of tasks, such as text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection.
- SVMs are adaptable and efficient in a variety of applications because they can manage high-dimensional data and nonlinear relationships.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.
- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below

diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example:

SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

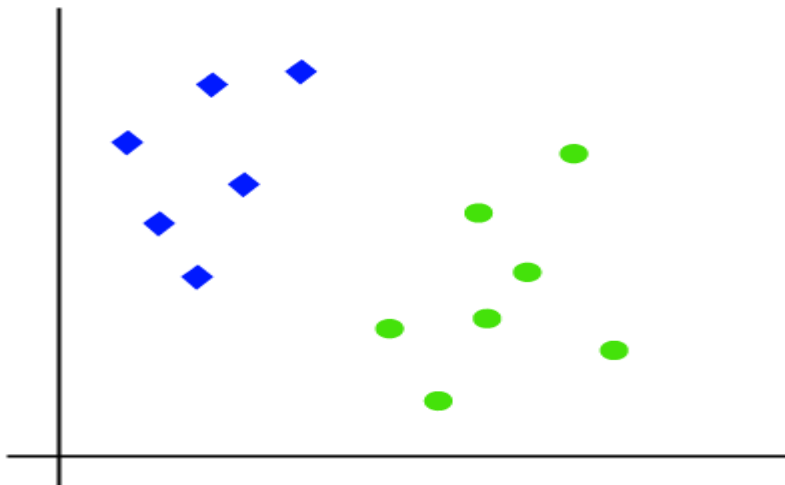
Support Vectors:

The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

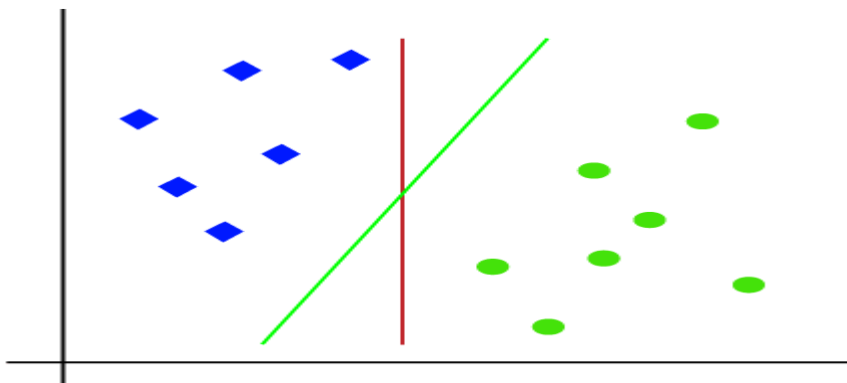
How does SVM works?

Linear SVM:

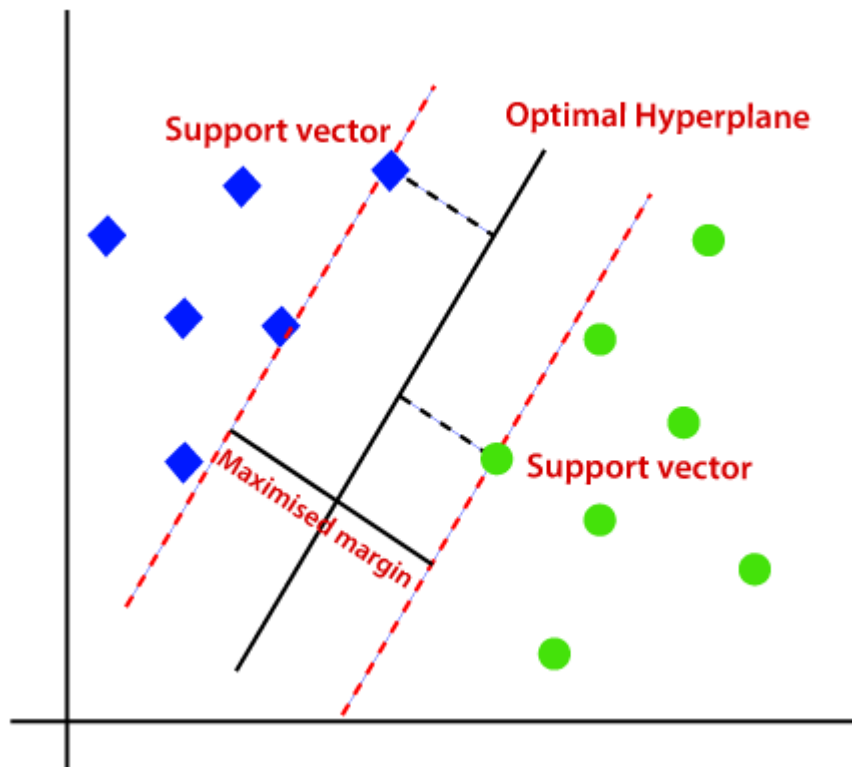
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

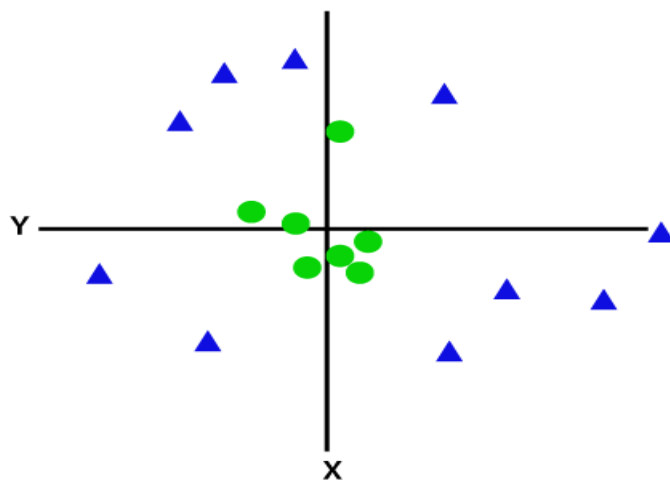


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a hyperplane. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as margin. And the goal of SVM is to maximize this margin. The hyperplane with maximum margin is called the optimal hyperplane.



Non-Linear SVM:

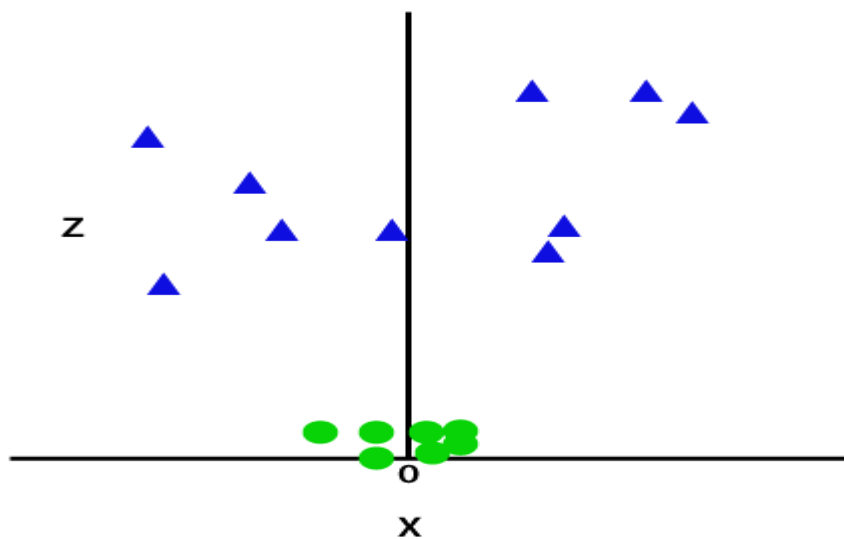
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



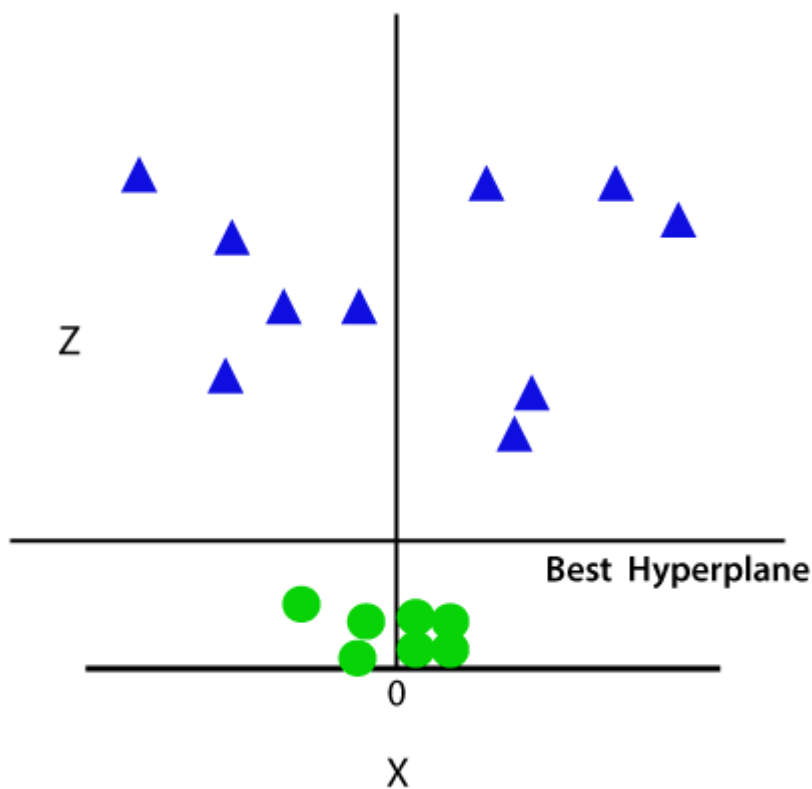
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z = x^2 + y^2$$

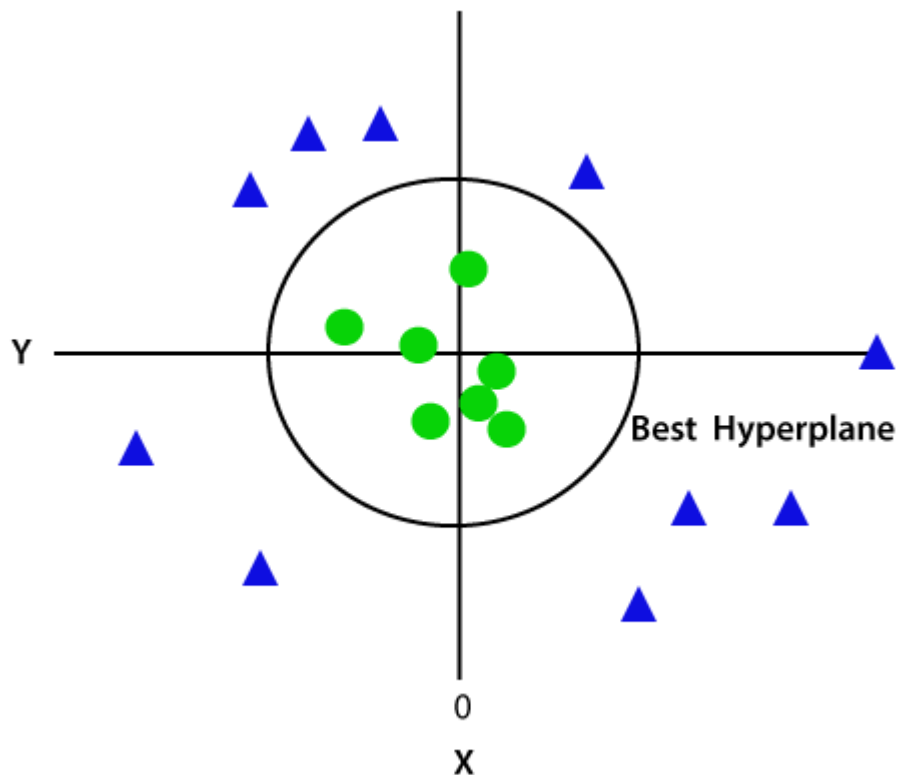
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis.
If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.